

Optimizing Partial Magic Squares

Anthony Kupshik, Clara Thompson, Eve Williamson
and Megan Sanderson

MA2760: Mathematical Investigations with Python
School of Mathematics,
Cardiff University.

April 15, 2025

1 Introduction

A magic square is a square array of numbers, where the sums of each row, column, and diagonal are equal. Their sum is known as the magic constant [1]. The cost of a magic square measures how far the square deviates from a valid magic square, and is defined by the sum of squared differences between the row, column, and diagonal sums, and the magic constant.

The partial magic square problem begins with an $n \times n$ grid in which some cells are pre-filled, and the rest are blank. The goal is to fill in the blanks to form a valid magic square. In this context, a cost of zero indicates a perfect solution.

This report investigates a computational approach to solve this problem using a local search algorithm. Starting from a randomized initial state, the algorithm iteratively refines the solution by minimizing the cost function, thereby approaching a valid magic square.

2 Functionality

The user inputs a text file where the first line is the size n of the magic square. The program creates an $n \times n$ grid filled with zeros, then reads the next n lines. Each line is split into individual strings converted to integers; non-zero integers are treated as fixed "given" values and placed in their corresponding positions while also being counted.

A randomized initial solution is generated by identifying the missing numbers from 1 to n^2 that aren't present in the fixed positions. It then randomly shuffles these missing values and fills each blank cell with one of the shuffled numbers. This process creates a complete, randomized solution that retains the original given values as shown below:

$$\begin{array}{c} \begin{bmatrix} 7 & 0 & 1 & 14 \\ 0 & 0 & 0 & 11 \\ 0 & 0 & 0 & 0 \\ 0 & 6 & 0 & 0 \end{bmatrix} \\ \text{Square from Text File} \end{array} \quad \rightarrow \quad \begin{array}{c} \begin{bmatrix} 7 & 10 & 1 & 14 \\ 2 & 13 & 12 & 11 \\ 4 & 15 & 5 & 8 \\ 3 & 6 & 16 & 9 \end{bmatrix} \\ \text{Randomized Initial Solution} \end{array}$$

To find a valid magic square, the algorithm repeatedly performs a swap between two non-fixed cells to generate a new solution and recalculates the cost. If the new solution yields a cost that is equal to or lower than the current cost, the swap is accepted, indicating an improvement in approaching the magic square solution; otherwise, the swap is undone, and the algorithm keeps searching.

$$\begin{bmatrix} 7 & 10 & 1 & 14 \\ 2 & 13 & 12 & 11 \\ 4 & 15 & 5 & 8 \\ 3 & 6 & 16 & 9 \end{bmatrix} \xrightarrow{15 \Leftrightarrow 3} \begin{bmatrix} 7 & 10 & 1 & 14 \\ 2 & 13 & 12 & 11 \\ 4 & 15 & 5 & 8 \\ 3 & 6 & 16 & 9 \end{bmatrix}$$

Cost: 612 Cost: 564

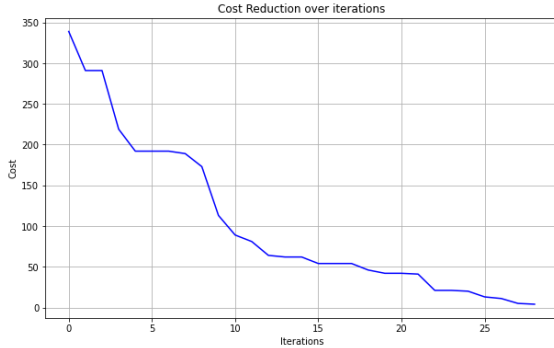
In this case, as $564 < 612$, the new solution would be accepted, and the algorithm will continue searching.

3 Enhanced Initialisation Strategies

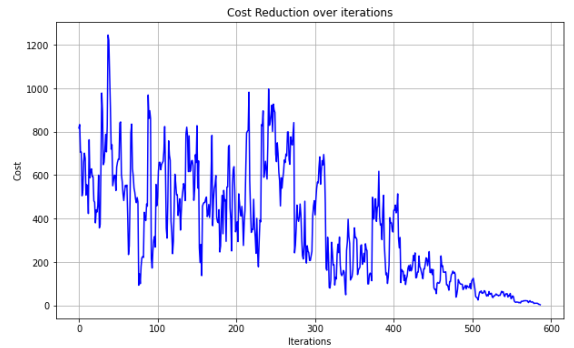
3.1 Simulated Annealing Algorithm

The simulated annealing algorithm[2] is inspired by annealing in metallurgy, a process where materials are heated and slowly cooled to remove defects.

The algorithm works very similar to our local search technique. The simulated annealing algorithm starts at a high temperature, allowing it to explore a wider range of candidate solutions. A worse candidate solution will have a higher cost than the current solution, and may be accepted with a probability of $P(accept) = \exp\left(-\frac{\Delta}{T}\right)$. This probabilistic acceptance helps the algorithm escape from local minima by occasionally accepting worse moves.



(a) Cost Reduction: Local Search



(b) Cost Reduction: Simulated Annealing

Figure 1: Comparison of Local Search and Simulated Annealing in Cost Reduction of 4x4 Zero Matrices

Our local search algorithm shows a consistent decline in cost, with the best solution being found after less than 30 iterations.

However, our simulated annealing algorithm shows an inconsistent decline in cost due to the altered acceptance criteria. This leads to the best solution being found after 600 iterations.

Our data therefore leads us to conclude that our local search algorithm is more efficient when compared to our simulated annealing algorithm.

3.2 Exploring Matrices of Different Sizes

The next part of our investigation involved the application of our local search technique to matrices of different sizes. We explored the effect of size on cost reduction and the success rate of finding an optimal solution.

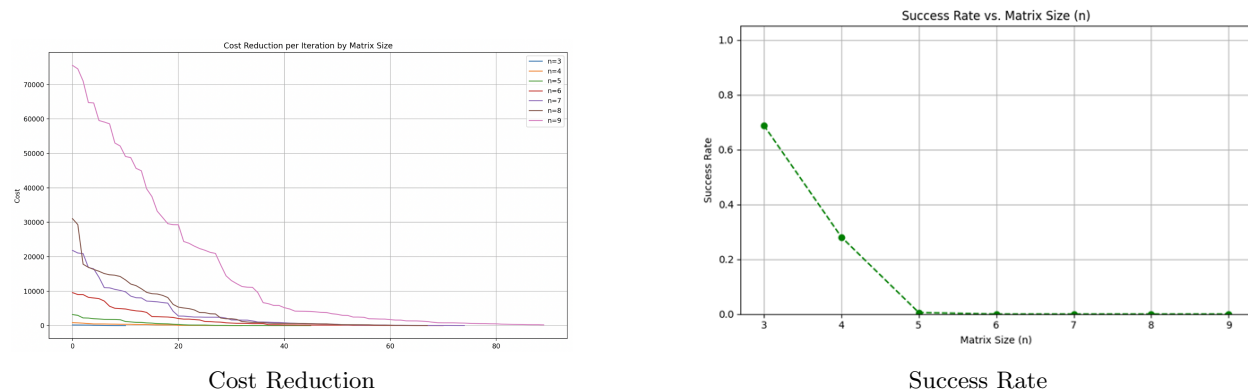


Figure 2: Comparison of Cost Reduction and Success Rates between Different Sized Matrices

The first graph shows that the larger the square, the higher its initial cost. We can also see that the higher the cost, the greater the number of iterations required to reach an optimal solution. This is due to the fact that the number of possible permutations of a square increases with its size, therefore the local search algorithm must execute the swapping of many more cells in order to find better candidate solutions.

The second graph shows that the success rate of finding an optimal solution is highest with 3x3 matrices. As the matrix size increases, the success rate of finding an optimal solution decreases, with very little to no optimal solutions being found in matrices of sizes 5x5 and above. This shows that our local search algorithm struggles to find a magic square for larger matrices, therefore it should not be used to determine whether there exists an optimal solution for larger squares.

3.3 Analysing the Effect of the Number of Givens

This section examines how the number of predefined values (givens) affects the performance of the magic square solver algorithm. By increasing the number of givens across 3x3, 4x4, and 5x5 matrices, the results reveal trends in success rates, computational cost, and iteration counts.

The results demonstrate the impact of the number of givens on the performance of a magic square solver algorithm for 3x3, 4x4, and 5x5 grids over 20 trials with a maximum of 1000 iterations. For the 3x3 grid, the success rate decreases significantly as the number of givens increases, with a peak success rate of 65% at zero givens and dropping to 0% beyond three givens. Similarly, for the 4x4 grid, the algorithm achieves a maximum success rate of 25% at zero and one given, but success rates decline sharply to 0% as givens increase. For the more complex 5x5 grid, no successful solutions are observed across all trials regardless of the number of givens. Across all grid sizes, average cost and iterations generally increase with the number of givens, reflecting greater difficulty in finding solutions. The average time per trial remains constant at approximately 0.01 seconds, suggesting that computational time is not a limiting factor within the set iteration cap. These findings highlight that increasing problem constraints (givens) significantly reduces solver efficacy, particularly for larger grids.

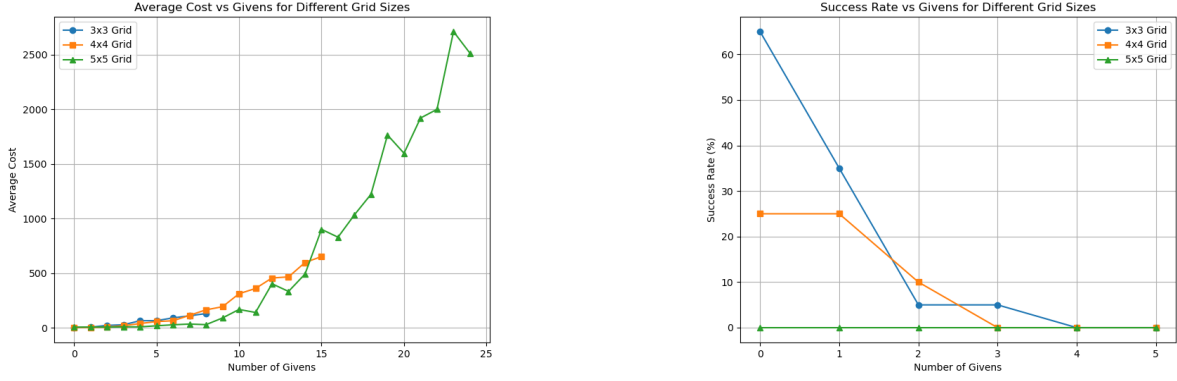


Figure 3: A comparison of the effect of the number of givens on average cost and success rate

3.4 Heuristics

To improve upon our local search technique, we looked into alternative initialisation strategies in an attempt to find optimal solutions for larger squares. This consisted of a simple heuristic method and a greedy constraint-based method.

Simple Heuristic Approach

The Simple Heuristic approach attempts to place each number in a position that minimises the deviation from the magic constant. This approach considers the current sum of each row and column when placing a number, choosing the value that minimises the combined deviation from the magic constant. This ideally creates a more balanced initial distribution leads to lower initial costs compared to random initialisation.

Greedy Approach

Building upon the heuristic method, the greedy approach introduces a more sophisticated framework for value placement. It prioritises cells based on how many constraints they influence, especially those within diagonals. It gives preference to nearly complete rows, columns, and diagonals, enabling more precise decisions. Additionally, its value selection approach adapts dynamically as more of the grid is filled, allowing more informed choices throughout the process.

3.5 Overall Analysis of Performance

We evaluated the effectiveness of these initialization strategies on magic square problems of various sizes with different numbers of givens. For each problem instance, we conducted 30 independent trials with a maximum of 500 iterations for 3×3 and 4×4 matrices, and 1000 iterations for 5×5 matrices. Key metrics included initial cost, final cost, and iterations required.

3.5.1 Performance of Different Heuristics Across Matrix Sizes

Table 1: Summary Statistics by Problem Size

Size	Metric	Random	Heuristic	Greedy
3x3	Average Initial Cost	119.34	117.23	89.10
	Average Final Cost	25.30	27.66	28.00
	Average Iteration	9.72	9.97	6.51
4x4	Average Initial Cost	733.07	674.60	582.00
	Average Final Cost	126.00	124.80	139.51
	Average Iteration	21.70	20.36	19.64
5x5	Average Initial Cost	2818.06	2236.84	2124.30
	Average Final Cost	258.06	310.67	235.51
	Average Iteration	40.59	38.82	37.80

Initial Cost Scaling: The data reveals a steep increase in initial costs as matrix size increases. From 3×3 to 4×4 , the cost increases roughly $6 \times$ for the Random approach, $5.8 \times$ for the Heuristic, and $6.5 \times$ for the Greedy approach. Further growth is observed from 4×4 to 5×5 , with increases of $3.8 \times$, $3.3 \times$, and $3.7 \times$, respectively. This exponential growth in initial costs aligns with the theoretical increase in problem complexity, as the number of constraints and possible arrangements grows significantly with matrix size.

Final Cost Scaling: Final solution costs also scale with size, increasing by approximately $5 \times$ from 3×3 to 4×4 for Random and Greedy strategies, and $4.5 \times$ for Heuristic. From 4×4 to 5×5 , the increase is roughly $2 \times$ for Random, $2.5 \times$ for Heuristic, and $1.7 \times$ for Greedy. This smaller scaling factor compared to initial costs indicates that the local search algorithm effectively mitigates some of the complexity growth.

Computational Effort: The number of iterations required grows consistently with matrix size. The transition from 3×3 to 4×4 results in a $2.2 \times$ increase for Random, $2 \times$ for Heuristic, and $3 \times$ for Greedy. From 4×4 to 5×5 , the growth is about $1.9 \times$ for all three strategies. This consistent scaling indicates that computational requirements roughly grows linearly with problem size, which is more favourable than the exponential growth in problem complexity.

Approach Effectiveness: In terms of initial solution quality, the Greedy approach consistently produces the best initial solutions, with costs 20–25% lower than those from the Random method across all sizes. This consistent advantage suggests that the heuristic principles underlying the Greedy approach remain effective regardless of problem scale.

For final solution quality, Random slightly outperforms others in 3×3 cases, Heuristic narrowly leads for 4×4 , and Greedy produces the best results for 5×5 . This suggests that as problem complexity increases, the benefits of a better initial solution (provided by Greedy) become more significant and harder to overcome through local search alone.

In terms of computational efficiency, Greedy consistently requires fewer iterations, although the margin narrows with size—from 33% fewer in 3×3 to 6.9% fewer in 5×5 . Suggesting that for very large problems, the choice of initialization approach may have less impact on computational efficiency.

3.5.2 Recommended Approach to the Partial Magic Square Problem

For 3×3 matrices, the Random approach offers a good balance between final quality and simplicity. For time minimisation, the Greedy approach achieves similar results with considerably fewer iterations.

In 4×4 matrices, the "Heuristic" approach delivers the best final outcomes with moderate computational effort. Considering that the gap between all strategies narrows, the choice is less critical. For 5×5 matrices,

the Greedy approach visibly outperforms the others in both final cost and efficiency. Overall, as matrix size increases, better initial solutions become more impactful.

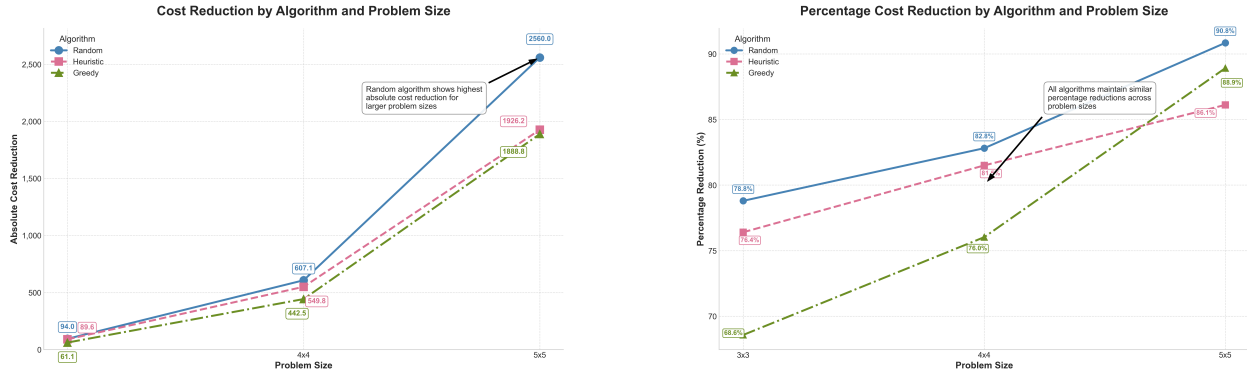


Figure 4: A comparison of absolute cost reduction with percentage cost reduction

Figure 3 illustrates the cost reduction achieved by each initialisation strategy across different problem sizes. While the Random approach shows the largest absolute cost reduction, particularly for larger matrices, this is primarily because it starts with significantly higher initial costs.

As shown in Table 1, the Greedy approach consistently achieves the lowest initial costs across all matrix sizes. Additionally, the "Greedy" approach produced the best final solutions for 5×5 matrices. This demonstrates that while Random exhibits greater cost reduction, the Greedy approach is much more efficient, especially as problem complexity increases.

4 Conclusion

In conclusion, this report explores the use of different algorithms and heuristics to find magic squares of various sizes. Our data showed that local search algorithm outperformed our simulated annealing algorithm in terms of efficiency, shown by the lower number of iterations required to reach an optimal solution.

Among the explored heuristic approaches, the "Greedy" approach consistently delivers the best performance, particularly in larger matrices (5×5), due to its superior initial quality and efficiency derived from our analysis. Although the "Random" approach achieves notable absolute cost reductions due to its high initial costs, it generally underperforms in final solution quality.

Conversely, the "Heuristic" approach shows balanced performance but becomes less competitive as complexity increases. Ultimately, the "Greedy" approach emerges as the recommended approach, especially as problem size and complexity grow, given its effective balance of initial solution quality and computational efficiency.

References

- [1] Wikipedia. Magic square. https://en.wikipedia.org/w/index.php?title=Magic_square&action=history&dir=prev. Accessed 03/25.
- [2] Wikipedia. Simulated annealing. https://en.wikipedia.org/wiki/Simulated_annealing. Accessed 03/25.